



SECURITY TESTING

ASSIGNMENT 3

Tran Dinh Hai - s4041605



**Lecturer:
Dr. Sreenivas Tirumala**

Table of Contents

1. Exploiting Web Services using SQL Injection Attacks	2
Q1.1: Log in to Ted's account without a password	2
Q1.2: Modify Bobby's salary	4
Q1.3: Modify Bobby's profile	7
Q1.4: Identify the vulnerable task queries	9
Q1.5: Display the maximum-task user's tasks	11
Appendix	15
Supporting Material	15

1. Exploiting Web Services using SQL Injection Attacks

Q1.1 In the above SQL statement, the variable `$input_uname` holds the string typed in the Username textbox, and `$hashed_pwd` holds the string typed in the Password textbox. User's inputs in these two textboxes are placed directly in the SQL query string. There is a SQL-injection vulnerability in the above `SELECT` query. Exploit the vulnerability and log into Ted's account without knowing the correct password. Demonstrate that your attack succeeds and explain your attack based on your exploited SQL.

The code snippet given in the assignment brief for this sub-question, taken from `unsafe_home.php`, is:

```
1 $input_uname = $_GET['username'];
2 $input_pwd = $_GET['Password'];
3 $hashed_pwd = sha1($input_pwd);
4 ...
5 $sql = "SELECT id, name, eid, salary, birth, ssn, address, email,
6 nickname, Password
7 FROM credential
8 WHERE name= '$input_uname' and Password='$hashed_pwd'";
9 $result = $conn -> query($sql);
```

Answer

Vulnerability analysis. Both values placed into `$sql` originate from the client and are never validated. `$input_uname` is read unmodified from `$_GET['username']`, and `$hashed_pwd` is the SHA-1 digest of `$_GET['Password']`. Hashing the password does not protect this statement: SHA-1 only transforms the *password* field into a fixed-length hexadecimal string, but the vulnerability lies in the neighbouring *name* column, where `$input_uname` is concatenated into the query completely unmodified. Whatever the password field contains, an attacker who controls the username field alone can still reshape the statement. Both variables are joined into the query by direct PHP string interpolation, with no parameterised query (an `mysqli` prepared statement with bound placeholders) and no input escaping (`mysqli_real_escape_string()`) applied to either one:

```
1 WHERE name= '$input_uname' and Password='$hashed_pwd'
```

The single quotes around each variable mark where the developer *intends* the string literal to start and end, but they do nothing to stop the value itself from containing a quote character. Because the query is assembled as plain text before MySQL ever parses it, a quote inside `$input_uname` is indistinguishable from the quote the developer typed around the variable. This is the defining trait of a string-based SQL injection in an authentication check: user input is interpreted as part of the query's *syntax* rather than as inert *data*, so whoever controls the input can rewrite the logic of the query, including removing the password predicate entirely.

Exploiting the vulnerability. I submitted Ted'# in the Username field and left the Password field blank, as shown in Figure 1.1. The browser transports the # character in the request line as the percent-encoded sequence %23; the web server decodes this back to the literal # before PHP ever reads `$_GET['username']`, so the value that reaches `$sql` is the raw character, not the encoded form shown in the address bar.

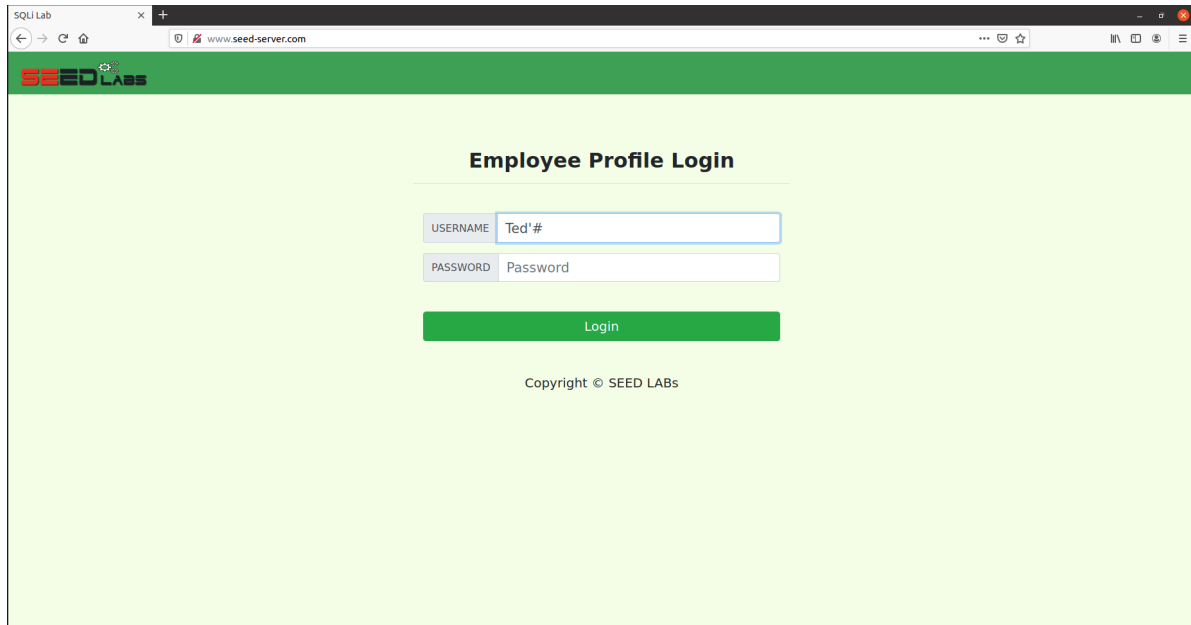


Figure 1.1. The injected username entered with the password field left blank.

With the Password field empty, `$hashed_pwd` is `sha1("")`. Substituting Ted'# for `$input_uname` therefore produces the raw statement, in which `$hashed_pwd` appears as its computed SHA-1 digest:

```
1 WHERE name = 'Ted'#' and Password='da39a3ee5e6b4b0d3255bfef95601890afd80709'
```

MySQL parses this left to right. Ted selects the intended account name. The apostrophe immediately after it is the one I injected: it closes the string literal that the developer opened with `name= '`, so the database compares name against the literal Ted only, not Ted'#. Parsing then reaches the # character. In MySQL, # begins a single-line comment that extends to the end of the statement, so everything after it – the stray closing quote left over from the original template, the literal and `Password=`, and the SHA-1 hash of the blank password field – is discarded before execution instead of being treated as SQL. The effective condition MySQL actually evaluates is therefore reduced to:

```
1 WHERE name = 'Ted'
```

This selects Ted's row by name alone; no password predicate participates in the query at all. In `unsafe_home.php`, a non-empty `$id` in the returned row is treated as a successful authentication: the script starts a session, stores Ted's identity in it, and calls `drawLayout()` to render his profile. Figure 1.2 confirms this outcome: the application displays the Ted Profile page populated with Employee ID 50000 and Salary 110000, the values recorded for Ted in `sql1lab_users.sql`,

confirming that the returned row genuinely belongs to Ted rather than to a different or empty account.

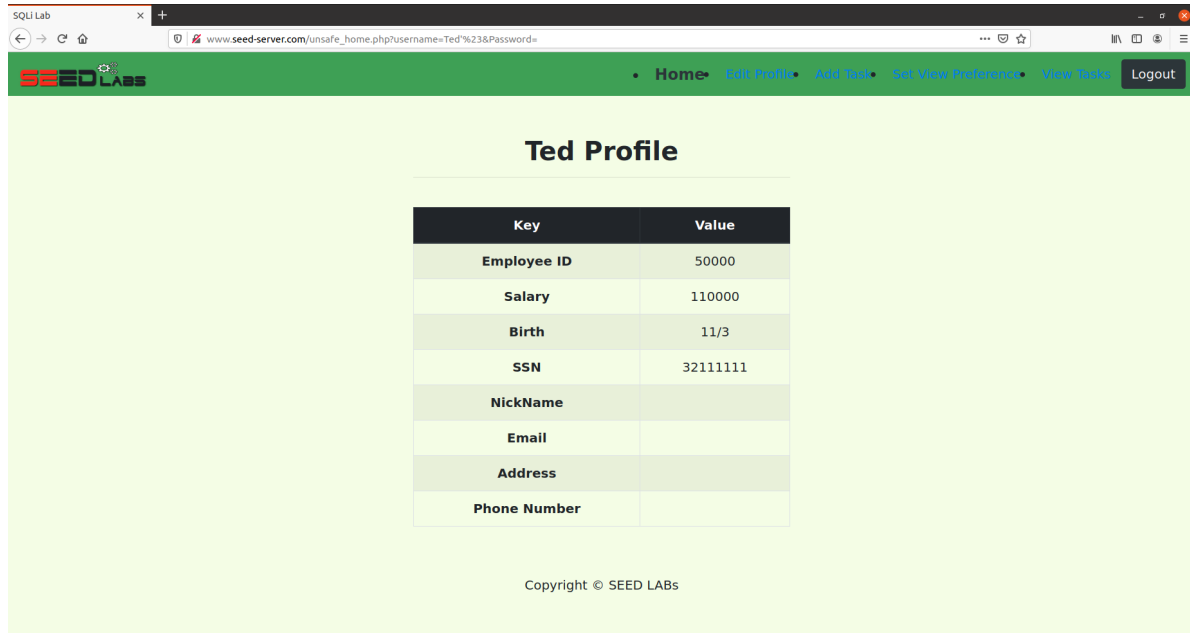


Figure 1.2. Ted's profile displayed after the injected login request.

This demonstrates that the login succeeded without knowledge of Ted's correct password: the SHA-1 comparison that should have enforced it was never evaluated, because the injected comment removed it from the executed query before MySQL had a chance to check it.

Q1.2 In above sub-question Q1.1, you have exploited the web application and logged in as Ted. Exploiting the vulnerability in this UPDATE SQL statement by using Ted's account to modify Bobby's salary to SALARY_1 without knowing Ted's password and Bobby's password. Demonstrate that your attack succeeds and explain your attack based on your exploited SQL.

The code snippet given in the assignment brief for this sub-question, taken from `unsafe_edit_backend.php`, is:

```

1  $hashed_pwd = sha1($input_pwd);
2  $sql = "UPDATE credential SET
3      nickname='$input_nickname',
4      email='$input_email',
5      address='$input_address',
6      Password='$hashed_pwd',
7      PhoneNumber='$input_phonenumber'
8  WHERE ID=$id;";
9  $conn->query($sql);
  
```

Answer

Vulnerability analysis. \$id is read from \$_SESSION['id'], so an attacker cannot set it directly through a request parameter – it is always the ID of whoever is currently logged in, Ted (ID=5) in this case. However, NickName, Email, Address, Password, and PhoneNumber are all read unmodified from \$_GET and concatenated into \$sql with no parameterised query and no escaping, exactly as in unsafe_home.php. Because the single quotes around each of these fields only mark the developer's *intended* string boundary, an attacker who breaks out of any one of them can append arbitrary SQL, including a brand-new WHERE clause, and comment away the original one. This means the attacker does not need to control \$id at all: redirecting the WHERE clause is enough to make the whole statement apply to any row, regardless of who is logged in.

The full snippet above is only executed when the Password field is non-empty. The actual file wraps it in a condition:

```

1  if($input_pwd!=''){
2      $hashed_pwd = sha1($input_pwd);
3      $_SESSION['pwd']=$hashed_pwd;
4      $sql = "UPDATE credential SET nickname='$input_nickname',email='$input_email',
              address='$input_address',Password='$hashed_pwd',PhoneNumber='$input_phonenumber'
              ' where ID=$id;";
5  }else{
6      $sql = "UPDATE credential SET nickname='$input_nickname',email='$input_email',
              address='$input_address',PhoneNumber='$input_phonenumber' where ID=$id;";
7  }
8  $conn->query($sql);

```

I left the Password field empty, so the executed statement is built by the else branch, which omits the Password assignment entirely. This branch is also never checked for success: \$conn->query(\$sql) is called and the script unconditionally redirects to unsafe_home.php whether the query succeeded or failed, so the application itself gives no on-page indication of the outcome.

Exploiting the vulnerability. While logged in as Ted from Q1.1, I opened Edit Profile and entered the following value in the NickName field, leaving Email, Address, and Phone Number at their pre-filled (blank) values and Password empty, as shown in Figure 1.3:

```

1  ', Salary=1605 WHERE ID=2 --

```

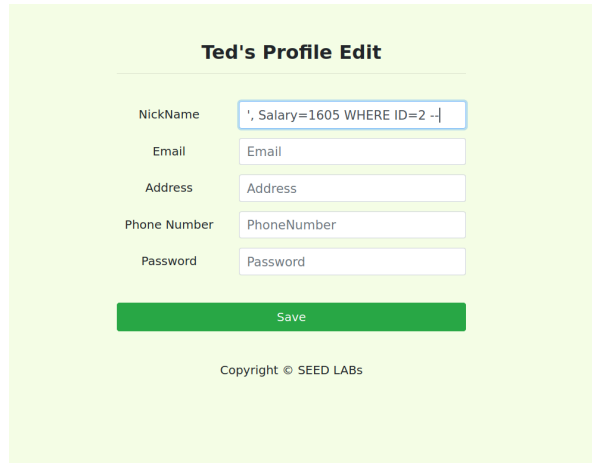


Figure 1.3. The injected NickName payload entered on Ted's Edit Profile form.

Substituting this into the else branch's \$sql, with Ted's session \$id equal to 5, produces the raw statement:

```
1 UPDATE credential SET nickname='', Salary=1605 WHERE ID=2 -- ',email='',address='',
  PhoneNumber='' where ID=5;
```

The leading apostrophe in the payload closes the nickname=''...'' literal immediately, giving nickname='' – a harmless no-op, since Bobby's NickName is already empty in the seed data. Everything after that quote is now parsed as SQL rather than string content: , Salary=1605 adds a second column assignment, and WHERE ID=2 supplies a brand new condition that targets Bobby instead of Ted. The trailing –, followed by a space, then opens a MySQL end-of-line comment that discards everything after it – the stray closing quote left over from the template, the remaining email/address/PhoneNumber assignments, and the real where ID=5 clause that would otherwise have kept the update on Ted's own row. The effective statement MySQL actually executes is therefore reduced to:

```
1 UPDATE credential SET nickname='', Salary=1605 WHERE ID=2
```

This updates Bobby's row (ID=2) alone; Ted's row is never touched, because the real where ID=5 clause was commented out before MySQL parsed it.

Since unsafe_edit_backend.php gives no visible success or failure message, I verified the result out of band by logging out and logging back in as Bobby using the same technique as Q1.1 (Bobby's # with the Password field left blank), which loads Bobby's profile directly from his row in credential. Figure 1.4 shows the result:

Boby Profile	
Key	Value
Employee ID	20000
Salary	1605
Birth	4/20
SSN	10213352
NickName	
Email	
Address	
Phone Number	

Copyright © SEED LABs

Figure 1.4. Boby's profile after the attack, showing Salary updated to 1605.

Boby's Salary field now reads 1605, matching SALARY_1 for student s4041605. Employee ID (20000), Birth (4/20), and SSN (10213352) are unchanged from `sql1lab_users.sql`, confirming this is genuinely Boby's row, and NickName remains blank, consistent with the injected `nickname=""` assignment being a no-op. This demonstrates that Ted's authenticated session was used to modify a different user's row without knowing Ted's password (already bypassed in Q1.1) or Boby's password at any point.

Q1.3 In above sub-question Q1.1, you have exploited the web application and logged in as Ted. Exploiting the vulnerability in this UPDATE SQL statement by using Ted's account to modify Boby's nickname to your student name, email to your RMIT student email, and address to "RMIT", without knowing Ted's password and Boby's password. Demonstrate that your attack succeeds and explain your attack based on your exploited SQL.

Answer

Vulnerability analysis. This sub-question exploits the same UPDATE statement identified in Q1.2, again via the `else` branch of `unsafe_edit_backend.php` that runs when the Password field is left empty:

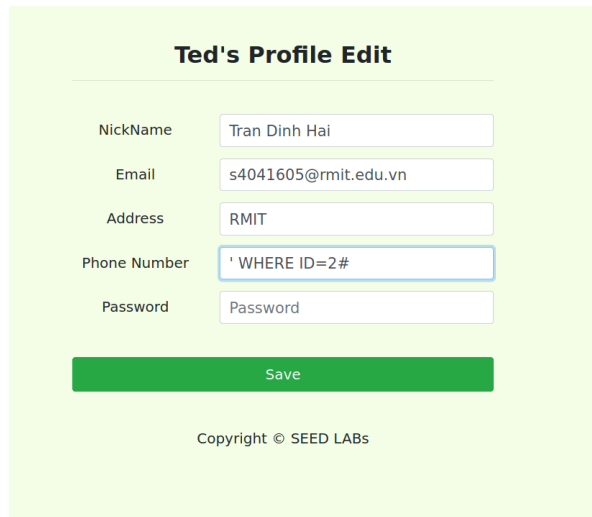
```
1 $sql = "UPDATE credential SET nickname='$input_nickname',email='$input_email',
    address='$input_address',PhoneNumber='$input_phonenumber' where ID=$id;";
```

In Q1.2 I broke out of the *first* field (NickName) and commented away everything after it, so the earlier, genuine-looking email/address/PhoneNumber values were never applied to any row – only the injected Salary assignment mattered. For this sub-question I need three fields (NickName, Email, Address) to actually take effect on Boby's row, so injecting through the first field will not work: anything before the injection point is discarded by the comment, and anything after it never executes on the intended row. Instead, I inject through the *last* field, PhoneNumber, immediately before the real WHERE clause. A single UPDATE statement has exactly one WHERE

clause governing every SET assignment in it, so replacing that clause at the last possible point makes *all* of the preceding assignments – including the legitimate-looking NickName, Email, and Address values – apply to whichever row the injected clause names, rather than to Ted’s own row.

Exploiting the vulnerability. While logged in as Ted, I opened Edit Profile and entered the following values, leaving Password empty so the else branch above is used, as shown in Figure 1.5:

- NickName: Tran Dinh Hai
- Email: s4041605@rmit.edu.vn
- Address: RMIT
- Phone Number: ' WHERE ID=2#



The screenshot shows a web form titled "Ted's Profile Edit". It contains several input fields: NickName (Tran Dinh Hai), Email (s4041605@rmit.edu.vn), Address (RMIT), Phone Number (' WHERE ID=2#), and Password (Password). A green "Save" button is at the bottom. The form is set against a light green background.

Figure 1.5. The injected values entered on Ted’s Edit Profile form, with the payload in Phone Number.

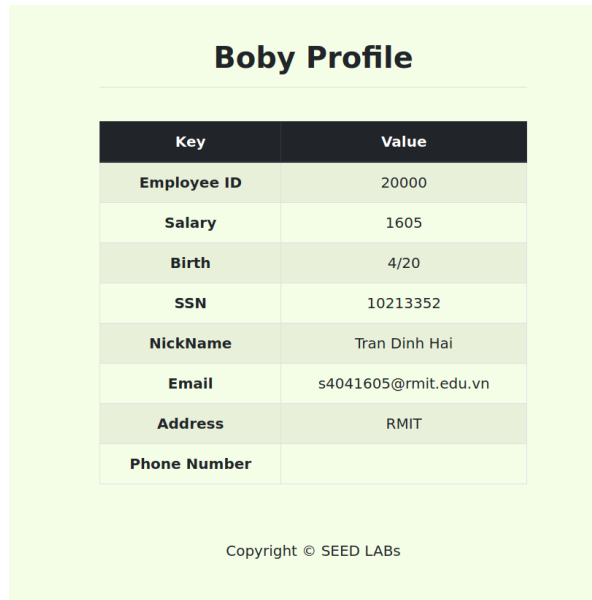
Substituting these into \$sql, with Ted’s session \$id equal to 5, produces the raw statement:

```
1 UPDATE credential SET nickname='Tran Dinh Hai',email='s4041605@rmit.edu.vn',address
  ='RMIT',PhoneNumber='' WHERE ID=2#' where ID=5;
```

The apostrophe in the Phone Number payload closes PhoneNumber='...' immediately, giving PhoneNumber='' – a no-op, since Bobby’s Phone Number is already blank in the seed data. WHERE ID=2 then supplies a brand-new condition for the statement, and # opens a MySQL comment that discards everything after it: the stray closing quote left over from the template and the real where ID=5 clause. Because this substitution replaces the statement’s only WHERE clause rather than being commented out itself, every assignment before it – nickname, email, and address – is retained and now applies to ID=2. The effective statement MySQL actually executes is:

```
1 UPDATE credential SET nickname='Tran Dinh Hai',email='s4041605@rmit.edu.vn',address
  ='RMIT',PhoneNumber='' WHERE ID=2
```

I verified the result the same way as Q1.2: logging out and back in as Bobby (Bobby's #, Password blank). Figure 1.6 shows the result:



Bobby Profile

Key	Value
Employee ID	20000
Salary	1605
Birth	4/20
SSN	10213352
NickName	Tran Dinh Hai
Email	s4041605@rmit.edu.vn
Address	RMIT
Phone Number	

Copyright © SEED LABs

Figure 1.6. Bobby's profile after the attack, showing the updated NickName, Email, and Address.

Bobby's profile now shows NickName Tran Dinh Hai, Email s4041605@rmit.edu.vn, and Address RMIT, exactly as submitted. Phone Number remains blank, consistent with the injected PhoneNumber="" assignment being a no-op. Salary still reads 1605 from Q1.2, and Employee ID (20000), Birth (4/20), and SSN (10213352) remain unchanged, confirming this is still genuinely Bobby's row. This demonstrates that Ted's authenticated session was again used to modify Bobby's row – this time three fields at once – without knowing Ted's password or Bobby's password at any point.

Q1.4 In the above two web pages, you need to identify the vulnerable SQL queries that are exploitable to allow the attacker to conduct the above attack: "display all the tasks of the user who has the maximum number of tasks when you view your tasks". Show the identifies vulnerabilities and explain why they are exploitable to allow for the attack.

Answer

This attack relies on two vulnerable statements working together: one that lets an attacker store arbitrary text unchecked, and one that later reads that text back and executes it as part of a new query.

```

1  $newfavourite = $_POST["favourite"] . " ";
2  $newfavourite.= $_POST["order"];
3
4  if($prefid!=""){
5      $sql2 = "UPDATE preference SET favourite='$newfavourite' where PreferenceID=
              $prefid;";
6  }else{

```

```

7     $sql2 = "INSERT INTO preference(favourite,Owner) VALUES ('$newfavourite', $id);
      ";
8   }
9   ...
10  $conn->query($sql2);

```

`$_POST["favourite"]` and `$_POST["order"]` are taken directly from the request with no validation – there is no check that favourite names a real column (Name, Hours, Amount, Description, Type) or that order is asc/desc. The two are concatenated into `$newfavourite` and written straight into the preference table's favourite column via string interpolation, with no escaping. This statement is itself injectable (an attacker could break out of the `'$newfavourite'` literal the same way as in Q1.2/Q1.3), but its more significant role here is that it lets an attacker persist *any* string, unchecked, for later use.

```

1  $sql = "select favourite from preference where owner=$id limit 1";
2  ...
3  $favourite = $json_a[0]['favourite'];
4  if($favourite==""){
5      $favourite = "hours desc";
6  }
7
8  $sql1 = "select tasks.Name as taskname, credential.Name as ownername, tasks.Hours,
          tasks.Amount,tasks.Description,tasks.Type
9  from tasks, credential where tasks.owner=credential.ID and tasks.owner=$id "
10 . "order by tasks." . $favourite ;
11
12 $queryList = explode(";", $sql1);
13
14 foreach($queryList as $subquery){
15     if(strlen($subquery)>0) {
16         if (!$tempResult = $conn->query($subquery)) {
17             die('There was an error running the query [' . $conn->error . ']\n');
18         }
19         ...
20     }
21 }

```

The stored favourite value is read back from preference and concatenated, unquoted and unescaped, directly after the literal text `order by tasks.` to build `$sql1`. Because it is never checked against a whitelist of column names, an attacker who controls its content controls part of the SQL text itself, not just a data value. Worse, the script then calls `explode(";", $sql1)` and executes every non-empty resulting fragment as its own independent query via `$conn->query($subquery)` inside the loop. This is a stacked-query mechanism built into the application: if favourite contains a `;` followed by a second complete SQL statement, that second statement is executed exactly as if the attacker had submitted it directly.

Why this is exploitable: the complete data flow. An authenticated user (an attacker needs no special privilege beyond a normal session) submits the Set View Preference form with crafted favourite/order values instead of the intended dropdown selection and asc/desc text. Query 1 persists this crafted string verbatim into that user's row in preference, with no validation. When the same user opens View Tasks, Query 2 reads that string back out of the database and splices it, still unvalidated, into a new SELECT immediately after order by tasks.. Because the resulting statement is split on ; and each fragment is executed separately, a favourite value of the form <valid column>; <second SELECT statement> causes the second statement to run as a genuinely separate query against the same database connection – for example, one that replaces the intended restriction tasks.owner=\$id with tasks.owner=userIdMaxTasks(), so it returns another user's tasks instead of the attacker's own. Query 1 is the injection point (no input validation on write); Query 2 is where the stored payload is later parsed apart and executed (no output validation on read, plus explicit multi-statement execution). Neither vulnerability alone completes the attack – Q1.5 exploits both together.

Q1.5 You need to exploit the identifies vulnerable SQL queries in above Q1.5 to conduct the attack: "display all the tasks of the user who has the maximum number of tasks when you view your tasks". Demonstrate that your attack succeeds and explain your attack based on your exploited SQL.

Answer

Step 1: submit the payload. This attack needs both vulnerable statements from Q1.4 to work together. Query 1 is where I plant the malicious text; Query 2 is where that text later gets executed. While logged in as Ted, I opened Set View Preference and filled in the form as shown in Figure 1.7:

- Sort field dropdown (favourite): Hours
- Asc or Desc field (order):

```

1  ;
2  select tasks.Name as taskname, credential.Name as ownername, tasks.Hours, tasks.
   Amount, tasks.Description, tasks.Type
3  from tasks, credential
4  where tasks.owner=credential.ID and tasks.owner=userIdMaxTasks()
  
```

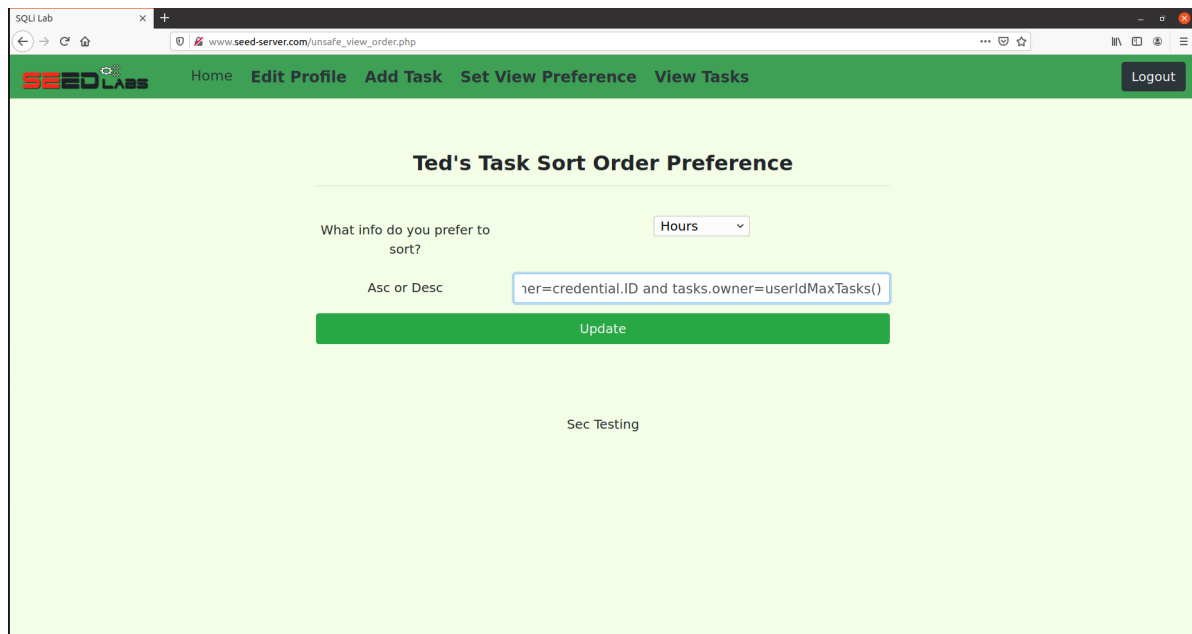


Figure 1.7. The Set View Preference form with Hours selected and the injected statement in the Asc or Desc field.

Picking a real column name (Hours) for the dropdown keeps the first half of the eventual statement valid SQL. The rest of the attack, a complete second SQL statement, is typed straight into the free-text order box. That field has no character restrictions and the form submits it as an ordinary POST request, so no proxy or browser console was needed to send it.

Step 2: how the payload is stored. Query 1 combines the two submitted fields into one string:

```
1 $newfavourite = $_POST["favourite"] . " ";
2 $newfavourite.= $_POST["order"];
```

With the values above, this builds:

```
1 Hours ;
2 select tasks.Name as taskname, credential.Name as ownername, tasks.Hours, tasks.
   Amount, tasks.Description, tasks.Type
3 from tasks, credential
4 where tasks.owner=credential.ID and tasks.owner=userIdMaxTasks()
```

Ted already had a row in preference, so this string is written into it unchanged by Query 1's UPDATE branch, with no check on its content.

Step 3: how the payload is executed. When View Tasks next loads for Ted, Query 2 reads that same string back out of the database as \$favourite and appends it to the end of a new query it is building. With Ted's session \$id equal to 5, the resulting \$sql1 is (shown here broken across lines for readability; PHP and MySQL both treat it as one continuous string, with no real line breaks):

```

1  select tasks.Name as taskname, credential.Name as ownername, tasks.Hours, tasks.
    Amount, tasks.Description, tasks.Type
2  from tasks, credential
3  where tasks.owner=credential.ID and tasks.owner=5
4  order by tasks.Hours
5  ;
6  select tasks.Name as taskname, credential.Name as ownername, tasks.Hours, tasks.
    Amount, tasks.Description, tasks.Type
7  from tasks, credential
8  where tasks.owner=credential.ID and tasks.owner=userIdMaxTasks()
  
```

The semicolon in the middle is the one I typed in the order field. Query 2 then runs `explode(";", $sql1)`, which cuts this single string into two separate pieces wherever a semicolon appears, and its foreach loop sends each non-empty piece to the database as its own, independent query. The table below explains what each of the two resulting queries does:

Query	Restriction on tasks.owner	What it returns
1: the original, legitimate half	tasks.owner=5 – Ted's own session ID, placed there by the application itself	Ted's own declared tasks, sorted by Hours
2: the half I injected	tasks.owner=userIdMaxTasks() – a function call I supplied, with no connection to who is logged in	The declared tasks of whichever user currently has the most tasks, joined with credential so that user's name is shown too

Step 4: the result. Figure 1.8 shows the View Tasks page after this request. It renders two separate tables, one for each query above. The first table lists Ted's own two tasks (*Back-end dev1*, 30 hours; *Front-end dev2*, 40 hours). The second table lists four tasks, and every one of them has Task Owner Alice (*On-site client requirement collection*, *Business Analysis*, *Demo App to client*, *Design System Architecture1*).

Ted's Declared Tasks

Task Name	Task Owner	Hours	Amount	Task Description	Type
Back-end dev1	Ted	30	7000	Dev back-end	Collaboration
Front-end dev2	Ted	40	11000	Dev mobile front-end	Collaboration

Task Name	Task Owner	Hours	Amount	Task Description	Type
On-site client requirement collection	Alice	50	10000	Collect client requirement	Collaboration
Business Analysis	Alice	50	4000	Task Breakdown	Individual
Demo App to client	Alice	50	2000	Onsite demonstration	Individual
Design System Architecture1	Alice	30	5000	Design components and communication protocols	Collaboration

Sec Testing

Figure 1.8. View Tasks after the attack, showing Ted's own tasks and, in the second table, Alice's tasks returned by the injected query.

The second table is the direct output of Query 2's injected half, so `userIdMaxTasks()` resolved to Alice's user ID in the database at the time of this request. This matches `sql1lab_users.sql`, where Alice (ID=1) is declared four tasks in the original seed data, more than any other user.

Conclusion. An attacker-controlled string, stored with no checks by Query 1, was later cut apart and run as a second, independent SQL statement by Query 2. That second statement ignored the restriction that should have limited results to the logged-in user and instead returned the declared tasks of the user with the most tasks overall, without that user's identity or credentials ever being entered anywhere in the attack. This is the complete attack the question asks for.

Appendix

Supporting Material

TODO *Add only supporting commands, configurations, or evidence that the Assignment 3 brief permits in an appendix. Remove this appendix if it is not required.*